

```
=====
=====
                                UNIVERSITY MANAGEMENT SYSTEM PROJECT REPORT
=====
=====
```

PROJECT NAME: Student Transcript and Management Portal
PROJECT TYPE: ASP.NET Core MVC Web Application
TARGET FRAMEWORK: .NET 8.0
DATABASE: PostgreSQL
DEVELOPMENT DATE: November 2024

```
=====
=====
1. PROJECT OVERVIEW
=====
=====
```

This is a comprehensive University Management System built using ASP.NET Core MVC.

The application provides a complete solution for managing university operations including student enrollment, course management, faculty administration, grading, attendance tracking, and transcript generation.

The system supports three main user roles:

- Administrator: Full system management capabilities
- Faculty: Course management, grading, and attendance tracking
- Student: Course registration, viewing grades, and accessing transcripts

```
=====
=====
2. TECHNOLOGY STACK
=====
=====
```

FRAMEWORK & RUNTIME:

- .NET 8.0 (Latest LTS version)
- ASP.NET Core MVC
- C# Programming Language

DATABASE:

- PostgreSQL (via Npgsql.EntityFrameworkCore.PostgreSQL)
- Entity Framework Core 8.0.4
- Code-First Migrations

AUTHENTICATION & AUTHORIZATION:

- ASP.NET Core Identity
- Role-Based Access Control (RBAC)
- Password requirements: Minimum 6 characters, requires digit, lowercase, uppercase

FRONTEND TECHNOLOGIES:

- Bootstrap (CSS Framework)
- jQuery (JavaScript Library)
- jQuery Validation (Form Validation)
- Razor Views (Server-Side Rendering)

3. PROJECT ARCHITECTURE

ARCHITECTURE PATTERN: MVC (Model-View-Controller)

PROJECT STRUCTURE:

├ Controllers/	- Application controllers handling HTTP requests
├ Models/	- Domain models and data entities
├ Views/	- Razor view templates
├ Data/	- Database context and initialization
├ ViewModels/	- View-specific data transfer objects
├ Migrations/	- Entity Framework database migrations
├ wwwroot/	- Static files (CSS, JS, images)
├ Properties/	- Application configuration files

4. DATABASE STRUCTURE

DATABASE NAME: UniversityDB
DATABASE PROVIDER: PostgreSQL

CORE ENTITIES:

1. ApplicationUser (Identity User)
 - Extends ASP.NET Core Identity
 - Links to Student or Faculty via foreign keys
 - Supports role-based authentication
2. Student
 - StudentId (Primary Key)
 - Name, Email, RollNumber
 - DepartmentId (Foreign Key)
 - One-to-one relationship with ApplicationUser
 - One-to-one relationship with Transcript
3. Faculty
 - FacultyId (Primary Key)
 - Name, Email
 - DepartmentId (Foreign Key)
 - One-to-one relationship with ApplicationUser
 - Can teach multiple CourseOfferings

4. Department
 - DepartmentId (Primary Key)
 - Name
 - Contains multiple Students, Faculty, and Courses
5. Course
 - CourseId (Primary Key)
 - Code (e.g., CSE101), Title, Credits
 - DepartmentId (Foreign Key)
 - Supports prerequisite relationships
6. Prerequisite
 - CourseId, RequiredCourseId (Composite relationships)
 - Defines course prerequisites
 - Prevents circular dependencies
7. Semester
 - SemesterId (Primary Key)
 - Name, StartDate, EndDate
 - Organizes course offerings by time period
8. CourseOffering
 - CourseOfferingId (Primary Key)
 - CourseId, FacultyId, SemesterId (Foreign Keys)
 - Represents a specific course instance in a semester
9. Enrollment
 - Composite Primary Key: StudentId + CourseOfferingId
 - EnrollmentDate
 - Grade (Letter Grade)
 - Component Scores: AssignmentScore, QuizScore, ProjectScore, MidScore, FinalScore, TotalScore
 - Grading breakdown: Assignments (10%), Quiz (10%), Project (10%), Midterm (20%), Final (50%)
10. Grade
 - GradeId (Primary Key)
 - Enrollment relationship
 - Detailed grade information
11. Attendance
 - AttendanceId (Primary Key)
 - Tracks student attendance for course offerings
 - Date and status information
12. Transcript
 - TranscriptId (Primary Key)
 - StudentId (Foreign Key, One-to-One)
 - CGPA (Cumulative Grade Point Average)
 - TotalCredits
 - Academic summary for students

IDENTITY TABLES:

- AspNetUsers, AspNetRoles, AspNetUserRoles, AspNetUserClaims,

AspNetRoleClaims, AspNetUserLogins, AspNetUserTokens

DATABASE MIGRATIONS:

1. 20251123062752_Init - Initial database schema
2. 20251127103158_InitialIdentitySetup - Identity system setup
3. 20251127133552_GradingAttendanceUpdate - Grading and attendance enhancements

=====

5. CONTROLLERS AND FUNCTIONALITY

=====

1. HomeController
 - Dashboard functionality
 - Role-based dashboard views (Admin/Faculty/Student)
 - Displays relevant statistics per user role
2. AccountController
 - User authentication (Login/Logout)
 - Handles user login functionality
3. AdminController
 - User management (Create Faculty, Create Student)
 - User listing and administration
 - System-wide administration tasks
4. StudentController
 - Student profile management
 - Course registration
 - View student details
 - Student-specific operations
5. FacultyController
 - Faculty dashboard
 - Manage grades for enrolled students
 - Track and manage attendance
 - Course offering management
6. CoursesController
 - Course CRUD operations
 - Course listing
 - Prerequisite management
 - Course creation and management
7. CourseOfferingsController
 - Create course offerings for semesters
 - Manage course instances
 - Link courses with faculty and semesters
8. DepartmentsController
 - Department CRUD operations
 - Department management

- Create, Edit, Delete departments
9. SemestersController
- Semester management
 - Create and edit semesters
 - Organize academic periods
10. TranscriptController
- Generate and view student transcripts
 - Display academic records
 - CGPA calculation and display

```
=====
=====
6. VIEWS AND USER INTERFACE
=====
=====
```

VIEW STRUCTURE:

Account Views:

- Login.cshtml - User authentication interface

Admin Views:

- CreateFaculty.cshtml - Create new faculty accounts
- CreateStudent.cshtml - Create new student accounts
- Users.cshtml - User management interface

Student Views:

- Index.cshtml - Student dashboard
- Details.cshtml - Student profile details
- Register.cshtml - Course registration

Faculty Views:

- Index.cshtml - Faculty dashboard
- ManageGrades.cshtml - Grade management interface
- Attendance.cshtml - Attendance tracking interface

Course Management Views:

- Courses/Index.cshtml - Course listing
- Courses/Create.cshtml - Create new courses
- Courses/Prerequisites.cshtml - Manage course prerequisites

Course Offerings Views:

- CourseOfferings/Index.cshtml - List course offerings
- CourseOfferings/Create.cshtml - Create course offerings
- CourseOfferings/Delete.cshtml - Delete course offerings

Department Views:

- Departments/Index.cshtml - Department listing
- Departments/Create.cshtml - Create departments
- Departments/Edit.cshtml - Edit departments
- Departments/Delete.cshtml - Delete departments

Semester Views:

- Semesters/Index.cshtml - Semester listing
- Semesters/Create.cshtml - Create semesters
- Semesters/Edit.cshtml - Edit semesters

Transcript Views:

- Transcript/Index.cshtml - View academic transcripts

Shared Views:

- _Layout.cshtml - Main layout template
- _Sidebar.cshtml - Navigation sidebar
- _LoginPartial.cshtml - Login/logout partial view
- _ValidationScriptsPartial.cshtml - Form validation scripts
- Error.cshtml - Error page

UI FEATURES:

- Responsive Bootstrap-based design
- Sidebar navigation with toggle functionality
- Mobile-friendly interface
- Form validation using jQuery Validation
- Role-based navigation and access control

=====

7. MODELS AND DATA STRUCTURES

=====

CORE MODELS:

- ApplicationUser - Identity user with role support
- Student - Student entity with enrollment tracking
- Faculty - Faculty entity with course offering management
- Department - Academic department organization
- Course - Course catalog entries
- Prerequisite - Course prerequisite relationships
- Semester - Academic time periods
- CourseOffering - Specific course instances
- Enrollment - Student-course enrollment with grading
- Grade - Detailed grade information
- Attendance - Attendance tracking records
- Transcript - Student academic summary

VIEW MODELS:

- LoginViewModel - Login form data
- AdminUserViewModels - Admin user management data
- TranscriptViewModel - Transcript display data
- DashboardViewModel - Dashboard statistics (defined in HomeController)
- ErrorViewModel - Error page data

RELATIONSHIPS:

- One-to-Many: Department → Students, Faculty, Courses
- One-to-Many: Course → CourseOfferings, Prerequisites
- One-to-Many: Faculty → CourseOfferings
- One-to-Many: Semester → CourseOfferings

- Many-to-Many: Student ↔ CourseOffering (via Enrollment)
- One-to-One: Student ↔ ApplicationUser
- One-to-One: Faculty ↔ ApplicationUser
- One-to-One: Student ↔ Transcript

8. SECURITY AND AUTHENTICATION

AUTHENTICATION SYSTEM:

- ASP.NET Core Identity framework
- Password-based authentication
- Session-based authentication

PASSWORD REQUIREMENTS:

- Minimum length: 6 characters
- Requires digit: Yes
- Requires lowercase: Yes
- Requires uppercase: Yes
- Requires non-alphanumeric: No

ROLES:

1. Admin - Full system access
 - User management
 - Department management
 - System configuration
2. Faculty - Teaching and grading access
 - Manage course offerings
 - Grade students
 - Track attendance
 - View assigned courses
3. Student - Academic access
 - Register for courses
 - View grades
 - Access transcripts
 - View enrollment status

DEFAULT ADMIN ACCOUNT:

- Email: admin@university.com
- Password: Admin@123
- Role: Admin

AUTHORIZATION:

- Role-based authorization on controllers and actions
- User-specific data filtering
- Protected routes and actions

9. DEPENDENCIES AND PACKAGES

```
=====
=====

NUGET PACKAGES:
```

1. Microsoft.AspNetCore.Identity.EntityFrameworkCore (8.0.4)
 - Identity system integration with Entity Framework
2. Microsoft.EntityFrameworkCore (8.0.4)
 - Core Entity Framework functionality
3. Microsoft.EntityFrameworkCore.Design (8.0.4)
 - Design-time tools for EF Core
4. Microsoft.EntityFrameworkCore.Tools (8.0.4)
 - Migration and scaffolding tools
5. Npgsql.EntityFrameworkCore.PostgreSQL (8.0.4)
 - PostgreSQL database provider for EF Core

```
FRONTEND LIBRARIES (via wwwroot/lib):
```

- Bootstrap - CSS framework
- jQuery - JavaScript library
- jQuery Validation - Form validation
- jQuery Validation Unobtrusive - ASP.NET integration

```
=====
=====

10. CONFIGURATION
```

```
=====
=====

CONNECTION STRING:
```

- Database: UniversityDB
- Host: localhost
- Port: 5432
- Username: postgres
- Provider: PostgreSQL

```
APPLICATION SETTINGS:
```

- Logging: Information level for general, Warning for ASP.NET Core
- Allowed Hosts: * (all hosts allowed)
- HTTPS Redirection: Enabled
- Static Files: Enabled
- Exception Handling: Configured for production

```
DATABASE INITIALIZATION:
```

- Automatic database seeding on application startup
 - Creates default roles (Admin, Faculty, Student)
 - Creates default admin user
 - Uses DbInitializer class for setup
- ```
=====
=====
```

## 11. KEY FEATURES

### ADMINISTRATIVE FEATURES:

- ✓ User account management (Create Faculty/Student)
- ✓ Department management (CRUD operations)
- ✓ Course catalog management
- ✓ Prerequisite management
- ✓ Semester management
- ✓ System-wide statistics dashboard

### FACULTY FEATURES:

- ✓ Course offering creation
- ✓ Student grade management
- ✓ Component-based grading (Assignments, Quizzes, Projects, Midterm, Final)
- ✓ Attendance tracking
- ✓ View assigned courses and students

### STUDENT FEATURES:

- ✓ Course registration
- ✓ View enrolled courses
- ✓ Grade viewing
- ✓ Transcript generation
- ✓ CGPA calculation
- ✓ Academic progress tracking

### ACADEMIC FEATURES:

- ✓ Course prerequisite system
- ✓ Semester-based course offerings
- ✓ Enrollment management
- ✓ Comprehensive grading system
- ✓ Attendance tracking
- ✓ Transcript generation

## 12. GRADING SYSTEM

### GRADING BREAKDOWN:

- Assignments: 10% of total grade
- Quizzes: 10% of total grade
- Projects: 10% of total grade
- Midterm Exam: 20% of total grade
- Final Exam: 50% of total grade
- Total: 100%

#### GRADE CALCULATION:

- Component scores stored as percentages (0-100)
- Total score calculated from weighted components
- Letter grade assigned based on total score
- CGPA calculated from all course grades

### 13. FILE STRUCTURE SUMMARY

CONTROLLERS: 10 files  
MODELS: 13 files  
VIEWS: 33+ Razor view files  
DATA: 2 files (DbContext, DbInitializer)  
VIEWMODELS: 2 files  
MIGRATIONS: 3 migration sets  
STATIC FILES: CSS, JavaScript, Bootstrap, jQuery libraries

### 14. DEVELOPMENT STATUS

PROJECT STATUS: Functional and Operational

#### COMPLETED COMPONENTS:

- ✓ Database schema design and implementation
- ✓ Identity and authentication system
- ✓ Role-based authorization
- ✓ CRUD operations for all major entities
- ✓ User interface with responsive design
- ✓ Grading and attendance systems
- ✓ Transcript generation
- ✓ Course prerequisite management

#### TECHNICAL NOTES:

- Uses Entity Framework Core Code-First approach
- Implements proper database relationships and constraints
- Follows MVC architectural pattern
- Implements separation of concerns
- Uses dependency injection throughout
- Includes proper error handling

### 15. TECHNICAL HIGHLIGHTS

#### ARCHITECTURE:

- Clean MVC architecture
- Repository pattern via DbContext
- ViewModels for data transfer
- Proper separation of concerns

#### DATABASE DESIGN:

- Normalized database schema
- Proper foreign key relationships
- Composite keys where appropriate
- Unique constraints on prerequisites
- Cascade delete restrictions where needed

#### CODE QUALITY:

- Nullable reference types enabled
- Proper async/await patterns
- Dependency injection
- Entity Framework best practices
- Identity integration

=====  
=====

END OF REPORT

=====  
=====

Report Generated: Based on project analysis  
Project Location: E:\WP-Project\WP-Project  
Framework Version: .NET 8.0  
Database: PostgreSQL

This report provides a comprehensive overview of the University Management System project, covering all major aspects including architecture, database structure, features, and technical implementation details.